Software Requirements Specification (SRS)
SurviveX

Team: Group 8
Authors: Aryan Bhagat, Praneet Chilla, Max Leavitt, Harry McGuire Jr, João Ziedins
Customer: Students in 6th - 8th grade
Instructor: Dr. James Daly

1 Introduction

This Software Requirements Specification (SRS) describes the design and functionality of SurviveX, an educational survival-adventure game built to help middle-school students strengthen their math skills through interactive gameplay. This document explains the purpose of the software, outlines the system's goals and constraints, and defines all major functional and non-functional requirements. It also provides diagrams and models that illustrate how core game systems operate, including exploration, combat, and math-based challenges.

1.1 Purpose

SurviveX is a 2D game made to help students in 6th - 8th grade improve their algebra, whether that be directly through students free will or because of teachers and parents. In SurviveX, students will solve algebra problems while exploring islands and surviving enemy encounters. The purpose of this SRS is to clearly describe the purpose of the game, define relevant terminology, give constraints and requirements, and describe the functionality of the game. This SRS will achieve that through a requirements document and various diagrams, such as use case, class, sequence, and state diagrams. Overall, the SRS will provide a clear blueprint of the game, allowing developers to properly map out the system functionality and teachers, parents, and students to verify if the software is a proper fit.

1.2 Scope

SurviveX is a 2D top-down educational survival game in which players explore randomly generated islands, manage resources, and engage in combat scenarios where success depends on solving math problems. The product integrates mathematics practice directly into its core gameplay loop, requiring students to solve curriculum based questions to progress, defeat enemies, and obtain rewards.

The primary objectives of the software are to provide math practice in an engaging, game-based environment for students in 6th - 8th grade, dynamically adapt math problem difficulty based on player performance, ensuring appropriate challenge levels, create a repeatable survival experience where exploration, combat, and resource management reinforce learning, offer clear feedback on answers and problem-solving accuracy.

1.3 Definitions, acronyms, and abbreviations

Armor - Upgradeable item that affects damage taken.

Weapon - Upgradeable item that affects damage dealt.

Inventory - Storage location for all player resources and equipment.

Island Challenge - A timed math problem sequence triggered on attempting to travel to a new island.

Chest - An object located near the middle of each island that rewards the player with resources when all zombies on the island have been defeated.

Coordinate - An (x, y) location representing a point in the game world.

Day/Night Cycle - A real-time loop affecting visibility, difficulty, and encounter rates.

Equipment - Items that improve player performance, including, weapons, armor, and upgrades.

Game Session - A playthrough instance

Health - Numeric value showing remaining endurance of player or raft

Loot - Resources or equipment obtained from island challenges or combat.

Night Vision Goggles - Upgrade allowing full daytime-level visibility at night.

Raft - Allows the user to travel the ocean.

Resources - Collectible materials: wood, scrap, and food.

Enemy - Hostile creatures that try to attack the player

Shark - An enemy in the ocean that the player will randomly engage with requiring timed math answers to inflict damage.

Zombie - An enemy on the islands that the player will engage with requiring timed math answers to inflict damage, rewarding loot upon defeating them.

Upgrade - Permanent improvements purchased with resources.

UI (User Interface) - All on-screen elements including HUD, inventory, math interface, and menus.

Visibility - Clarity of world and UI elements, affected by time of day or night vision goggles.

2D - Two-dimensional graphics, like viewing the game from above

Damage - How much health the player/raft loses from an attack

Input field - Where the players enters their answer to the questions

Sword - A type of weapon

Game Over - Appears when the player or raft health reaches zero. It forces the player to restart the game.

Spawn - Putting players or enemies onto the game world.

1.4 Organization

This document contains six more sections. Section 2 gives an overview of the game and how it works. Section 3 lists all the specific requirements in detail. Section 4 contains use cases, class, sequence, and state diagrams and explanations of each component of the diagrams to describe how the system works. Section 5 describes our prototype, how to run the prototype, and examples of in game content. Section 6 lists our references. Section 7 gives our point of contact.

2 Overall Description

This section provides a high-level overview of SurviveX, describing how the game fits within its intended environment, the major functions it performs, and the important characteristics of its users. It also identifies the system's constraints, assumptions, and elements that fall outside the scope of the current version. While later sections detail specific requirements, this section explains the broader context needed to understand how different components of SurviveX work together.

2.1 Product Perspective

Math is a key part of the world around us and students will need a strong foundation in it to be successful in school. However, many students struggle with the traditional lecture approach to learning and find it to be unengaging. The goal of SurviveX is to make 6th - 8th grade students excited about learning through an educational game and help reinforce math concepts that they've learned in class.

SurviveX runs locally and does not interact with external servers or databases. All progression, enemy generation, and math logic occur within the game engine. The system requires a keyboard for basic movement, typing math answers, and interacting with some menus. A mouse or trackpad will be needed for selecting UI elements and to navigate menus, although the game does not require precise mouse gameplay. A standard computer monitor displays the game. SurviveX runs on common desktop operating systems. The game engine handles rendering, input processing, and audio output without external dependencies. The device must have sufficient memory to run a 2D game with resource tracking, enemy behaviors, and island data. No long-term storage requirements exist, and no site-specific adjustments or configurations are required. The game runs as-is on supported devices.

2.2 Product Functions

SurviveX combines survival game mechanics with math practice through several main functions. The world generation system creates 4 unique islands each game with different biomes and resources to discover. Players explore using top-down 2D movement. The core feature is the math problem system, which presents 6-8th grade problems covering linear equations, systems of equations, functions, and geometry. These problems pop up when students discover new islands or encounter sharks. Students have a time limit to solve each problem and get immediate feedback, with difficulty adjusting based on their performance.

For gameplay, there's a resource management system where players collect five types of resources, which are wood, scrap, rope, food, and water, that show up in the HUD and inventory. The shop lets players upgrade rafts, weapons, and armor, as well as buy permanent upgrades. Combat involves enemies like zombies and sharks in turn-based encounters where solving math problems determines the outcome. The game also has a day/night cycle that affects visibility and enemy difficulty. Finally, the UI handles all the menus, HUD displays, inventory screens, and settings for adjusting audio, graphics, and gameplay options.

2.3 User Characteristics

The primary users are 6-8th grade students with varying math skill levels from below grade level to advanced, who possess basic computer literacy and keyboard/mouse proficiency. Students

have mixed gaming experience ranging from novices to experienced gamers familiar with 2D adventure games, and they expect entertainment value rather than traditional educational software. They are motivated by visible progress. Some students may have learning differences requiring accessibility accommodations such as larger text, extended time limits, or colorblind-friendly visual elements. Secondary users are educators with moderate computer proficiency who need to monitor student progress, verify curriculum alignment, and customize difficulty settings to match classroom needs.

2.4 Constraints

SurviveX must operate on standard classroom and home computers with limited processing power, requiring the game to maintain smooth performance while handling continuous movement, enemy encounters, and the day/night cycle without relying on high-end hardware or network connectivity. All math problem generation, resource tracking, combat calculations, and difficulty adjustments must be performed locally and consistently to ensure predictable game behavior for educational use. Because the target audience consists of middle-school students, all UI elements must remain simple, readable, and easy to navigate, with clear feedback during problem-solving and combat. The system must also ensure that player and raft health values, enemy damage, inventory updates, and timed questions behave reliably and cannot desynchronize or freeze, as these are core to the survival loop. Additionally, math content must remain aligned with grade-level standards and adapt within controlled limits so that problems do not become too difficult or trivial for student players.

2.5 Assumptions and Dependencies

SurviveX assumes that users will access the game on a desktop or laptop computer equipped with a functional keyboard and connected to a functional monitor, as well as the ability to display 2D graphics without advanced rendering capabilities. It is assumed that players can read English instructions, understand middle school level math concepts, and interact with simple user interfaces using keyboard input and mouse controls. The game depends on the underlying operating system to support standard file handling, audio playback, input management, and windowed or fullscreen display modes, as well as the game engine's ability to manage scenes, animations, timers, and collision detection consistently. Since SurviveX is designed to run offline, it is also assumed that no network access will be required for gameplay, and that the environment in which the game is used, such as a classroom or home setting, will provide enough focus and stability for players to complete timed math problems and interpret visual feedback correctly.

2.6 Apportioning of Requirements

There are several features that could be added in future iterations of SurviveX but fall outside the scope of the current release and the time allotted for this project. These include additions such as multiplayer or cooperative gameplay, expanded enemy types and environmental hazards, more diverse island biomes, larger crafting or base building systems, and enhanced analytics or teacher dashboards for tracking student performance. Additionally, a mobile or web-based version and more advanced customization options, such as teacher defined difficulty ranges or problem categories, could also be introduced. These elements are not included to ensure the initial development remains focused on delivering the core educational and gameplay objectives effectively while maintaining stability and meeting the constraints of the target hardware and classroom environments.

3 Specific Requirements

1. The game will generate grade-appropriate math problems aligned with 6-8th grade standards
    1.1. The game will show one of 4 different types of problems
        1.1.1. The game will create linear equation problems (ax + b = c format)
        1.1.2. The game will create systems of equations problems
        1.1.3. The game will create geometry problems (area, perimeter, volume)
        1.1.4. The game will create percentage and ratio problems
    1.2. The game will present math problems through an interactive UI overlay
    1.3. The game will provide immediate feedback on whether the student answered the question correctly
2. The game will contain resources
    2.1. The game will contain these resources types: wood, scrap, and food
    2.2. Resources are stored in an inventory location which can be accessed at anytime by the player
3. The game will contain four explorable islands
    3.1. The first island will contain a shop where players can purchase and upgrade equipment
        3.1.1. Equipment will be purchased with wood and scrap
        3.1.2. The player can purchase weapons, armor, and upgrades
    3.2. Each island will contain a set amount of zombies that need to be defeated before unlocking the next island
        3.2.1. Players will engage with zombies by running into them or by zombies running into them
        3.2.2. The player must answer timed math questions while engaged with the zombie
            3.2.2.1. If the player answers incorrectly or not in time, the zombie will deal damage to the player
            3.2.2.2. If the player answers correctly, the player will deal damage to the zombie
        3.2.3. Defeating a zombie will award resources to the player
        3.2.4. After the first island, defeating every zombie will spawn a chest in the center of the island that contains resources
        3.2.5. To complete the game, the player must defeat every zombie on every island
    3.3. Each island will contain a dock used to travel to other islands
        3.3.1. Islands 2 - 4 will be locked off in the beginning
        3.3.2. The player must correctly answer a slope problem before travelling to a new island

3.3.2.1. If the player incorrectly answers the problem, they will have to fight a shark
3.3.2.1.1. The player must answers timed math questions while engaged with the shark
3.3.2.1.1.1. If the player answers incorrectly or not in time, the shark will deal damage to the raft
3.3.2.1.1.2. If the player answers correctly, the player will deal damage to the shark

3.4. The game will trigger timed math challenges when the player enters a new island
3.4.1. The player will have the option to decline the challenge with no penalty
3.4.2. The player will be awarded resources if the challenge is completed in time
3.4.3. The player will lose resources if the challenge is not completed in time

4. The game will end if player health or raft health reach 0
4.1. Food can be used to restore player health
4.2. Wood can be used to restore raft health

5. The game will have a day/night cycle
5.1. The cycle corresponds to an in-game clock loop
5.2. Map visibility will decrease at night
5.3. Zombies and sharks deal more damage at night

6. The game will display all necessary information only during the day
6.1. The game will display all resource quantities and equipment
6.2. The game will show the current day and time
6.2.1. Sharks and zombies deal more damage as the day counter goes up
6.3. The game will show math problem text, input field, and allow the player to submit their answer

7. The game will contain different types of equipment
7.1. Equipment is stored in an inventory location which can be accessed at anytime by the player
7.2. The game will contain weapons
7.2.1. Better weapons will deal more damage
7.3. The game will contain armor
7.3.1. Better armor will reduce damage taken
7.4. The game will contain upgrades
7.4.1. Upgrades will consist of night vision goggles
7.4.1.1. Night vision goggles will improve the game world and UI visibility at night allowing the user to see as if it was daytime
7.4.2. Upgrades will last permanently once obtained
7.5. Players will start the game with only a sword

8. The game will play background music while the player explores the world

4 Modeling Requirements

This section presents comprehensive diagrams that illustrate the architecture and behavior of SurviveX. The diagrams include a use case diagram, a class diagram with detailed descriptions, two sequence diagrams, and a state diagram. Each diagram is accompanied by explanatory text to clarify the system's design and interactions.

4.1 Use Case Diagram

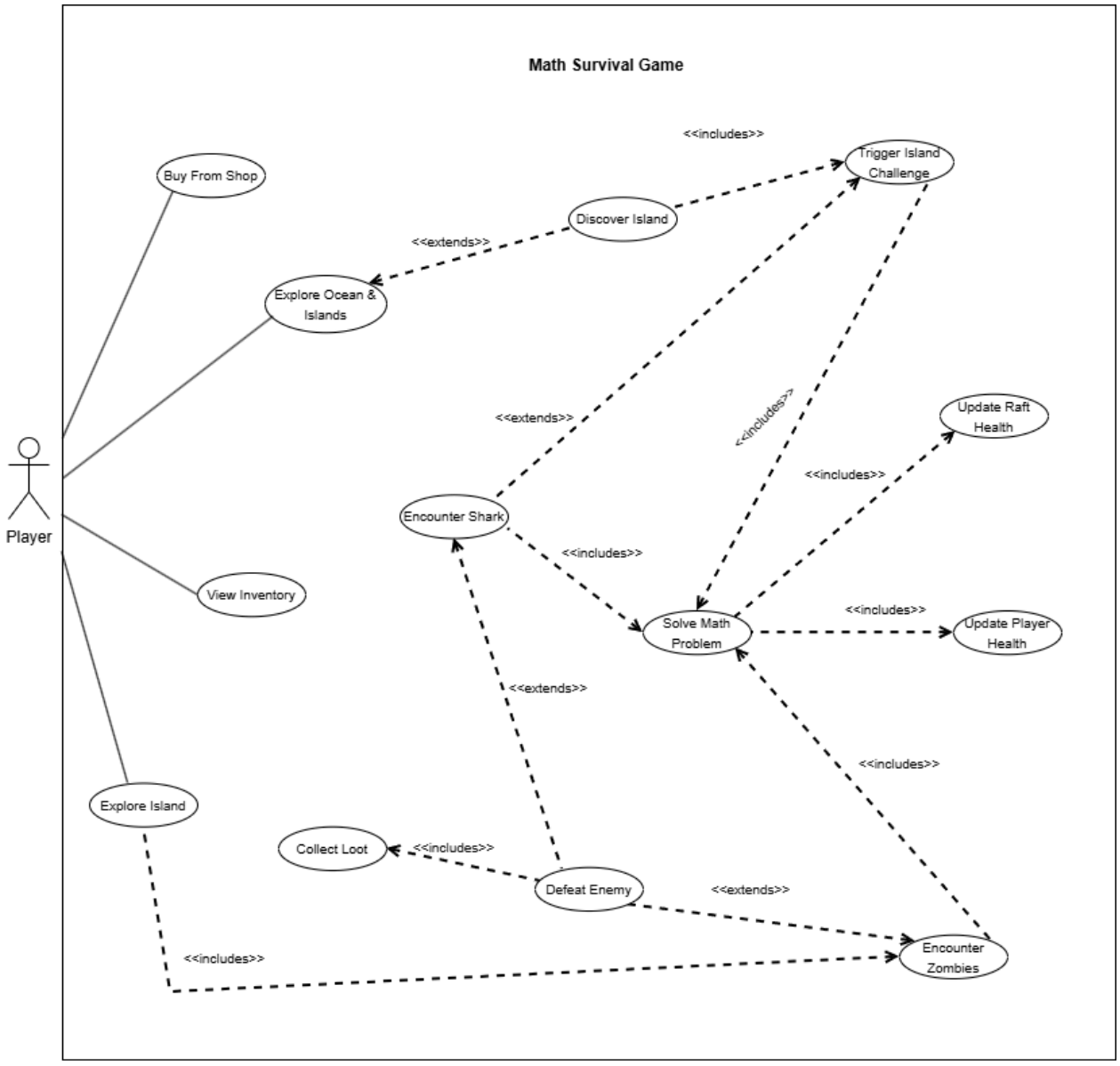

Figure 1: Use Case Diagram

Figure 1 above describes the primary interactions available to players within SurviveX. Each oval represents a distinct action or functionality, with solid lines connecting them to the actors

who perform these actions and dashed lines indicating relationships between use cases. The extends relationship denotes conditional variations of a use case, while includes signifies shared functionality that occurs when the previous event happens.

The diagram centers around exploration and survival mechanics. Players navigate through ocean environments, encounter various threats, and must utilize their math problem-solving skills to overcome challenges. The core gameplay revolves around resource management, equipment upgrades, and strategic decision-making. Combat encounters with zombies and sharks trigger the problem-solving system, which is the educational component of the game. Successful problem solving rewards players with resources and equipment, creating a feedback loop that encourages continued learning and engagement.

Use Case Descriptions

Use Case Name:	Explore Ocean & Islands
Actors:	Player
Description:	Players answer questions in order to travel to other islands on the raft, discovering new islands and encountering various challenges throughout their journey.
Type:	Primary
Includes:	Discover Island
Extends:	None
Cross-refs:	Requirement 3
Uses cases:	This is the central gameplay loop that encompasses all major player activities during a session.

Use Case Name:	Encounter Shark
Actors:	Player, System
Description:	During ocean travel, sharks will attack a player's raft if the coordinates are wrong, requiring immediate mathematical problem-solving to defend against damage.
Type:	Primary
Includes:	Solve Math Problem
Extends:	Explore Ocean & Islands
Cross-refs:	Requirement 3.3.2.1

Uses cases:	Represents the threat during ocean navigation that tests mathematical skills.

Use Case Name:	Discover Island
Actors:	Player
Description:	Upon arriving at an island, players may choose to disembark and explore its terrain, potentially triggering challenges and enemy encounters.
Type:	Primary
Includes:	Encounter Zombies
Extends:	Explore Ocean & Islands
Cross-refs:	Requirements 3.2, 3.4
Uses cases:	Gateway to island based content and progression.

Use Case Name:	Trigger Island Challenge
Actors:	Player, System
Description:	Islands present optional mathematical challenges that, when accepted and completed successfully, provide resource rewards to aid survival.
Type:	Primary
Includes:	Solve Math Problem
Extends:	Discover Island
Cross-refs:	Requirement 3.4
Uses cases:	Voluntary engagement mechanism that offers additional loot for players.

Use Case Name:	Encounter Zombies
Actors:	Player, System
Description:	Islands spawn hostile zombie enemies that players must confront, requiring mathematical problem-solving to defeat them and claim rewards.
Type:	Primary
Includes:	Solve Math Problem

Extends:	None
Cross-refs:	Requirement 3.2
Uses cases:	Primary combat mechanic on islands that combines threat with educational content.

Use Case Name:	Defeat Enemy
Actors:	Player, System
Description:	Players engage in combat by solving mathematics problems within time limits; correct solutions inflict damage while failures result in harm to the player or raft.
Type:	Primary
Includes:	Collect Loot
Extends:	Encounter Zombies, Encounter Shark
Cross-refs:	Requirements 3.2, 3.3.21
Uses cases:	Core combat resolution mechanism that directly links mathematical performance to survival.

Use Case Name:	Solve Math Problem
Actors:	Player, System
Description:	The system presents grade-appropriate mathematical problems that players must solve within specified time constraints, with immediate feedback provided.
Type:	Primary
Includes:	None
Extends:	None
Cross-refs:	Requirement 1
Uses cases:	Central educational component that appears in all combat and challenge scenarios.

Use Case Name:	Update Player Health

Actors:	System
Description:	The system modifies player or raft health values based on combat outcomes, resource availability, and environmental conditions.
Type:	Secondary
Includes:	Display Player Health
Extends:	Encounter Zombies
Cross-refs:	Requirements 3.2.2, 3.3.2.1.1, 4
Uses cases:	Maintains accurate player health tracking through all battles.

Use Case Name:	Collect Loot
Actors:	Player
Description:	Players acquire resources from defeated zombies, island challenges, and chests
Type:	Primary
Includes:	Manage Resources
Extends:	Solve Math Problems
Cross-refs:	Requirements 2, 3.2.3, 3.4
Uses cases:	Reward mechanism that gives loot earned by the player.

Use Case Name:	View Inventory
Actors:	Player
Description:	Players access their inventory interface to examine current resource quantities and equipment
Type:	Primary
Includes:	None
Extends:	None
Cross-refs:	Requirements 2.2, 7.1
Uses cases:	Information access point for resource management.

Use Case Name:	Update Raft Health
Actors:	System
Description:	The system modifies the raft's health based on shark attacks, resource depletion rates, or player-initiated repairs.
Type:	Secondary
Includes:	None
Extends:	None
Cross-refs:	Requirements 3.3.2.1.1.2, 4
Uses cases:	Maintains accurate raft health tracking through all battles.

4.2 Class Diagram

The class diagram presented below defines the structural foundation of SurviveX. Each class box contains the class name, its attributes with data types, and its operations. The diagram illustrates how different components of the game interact through various relationships including associations, compositions, and inheritance hierarchies.

The architecture centers on the Player class, which serves as the primary interface between the user and the game world. The GameEngine class acts as the central controller, managing game state and coordinating between different systems. The ProblemSystem provides the educational component by generating and evaluating mathematical problems. Combat is handled through the abstract Enemy class with concrete implementations for Zombies and Sharks. Resource management flows through the Inventory class, while progression occurs through the Shop class and various Equipment implementations. The SeaTravel class manages movement between islands, and the Island class contains the content and challenges players encounter during exploration.

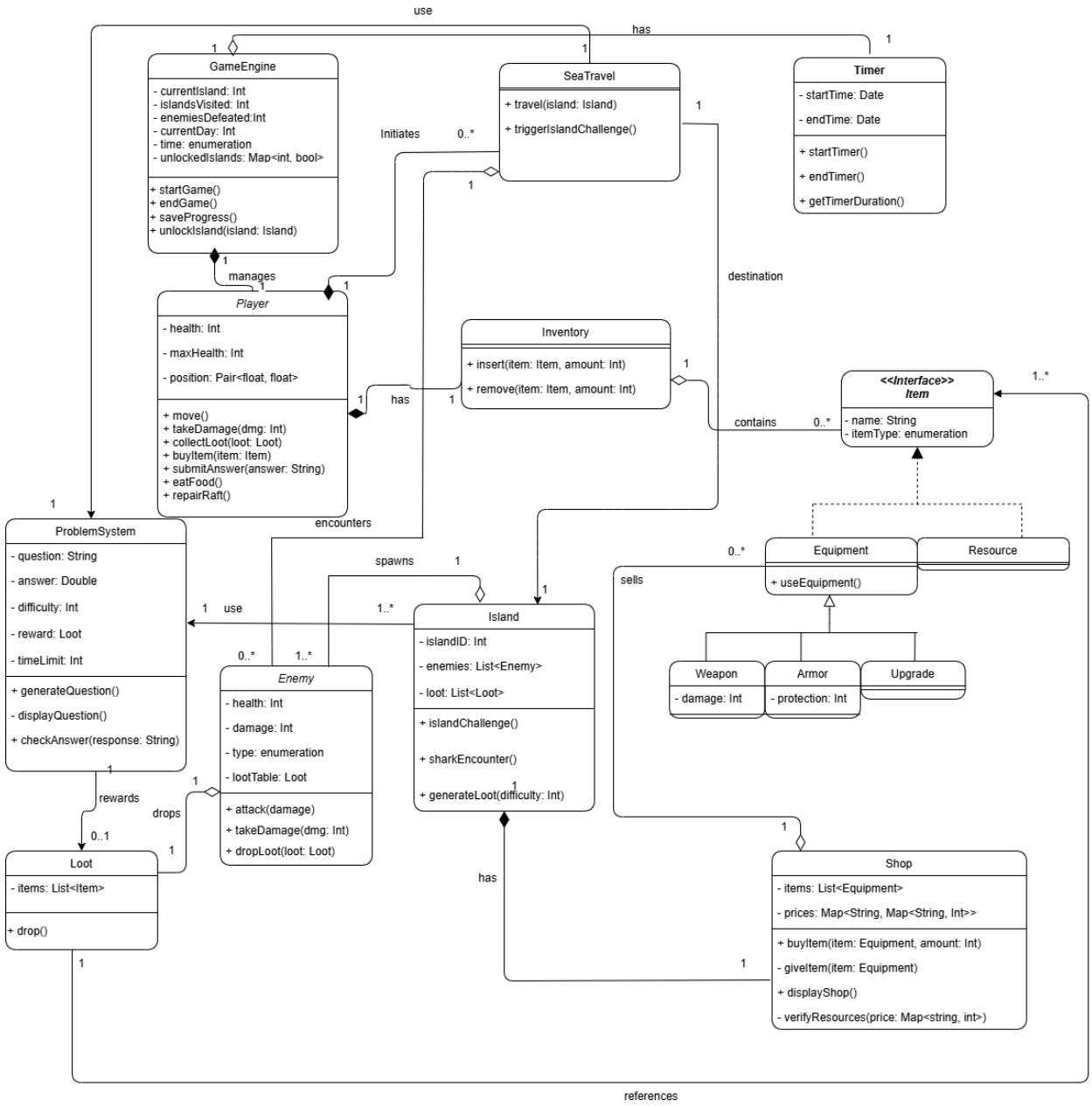

Figure 2: Class Diagram

Class Descriptions

Class: Player	
Description	Represents the player character with all associated attributes and actions available during gameplay.
Attributes	
Name	**Description**

health: Int	Current health points of the player character
position: Pair<float, float>	Player's current coordinates within the game world
maxHealth: Int	Maximum health points of the player character
Operations	
move()	Updates player position based on input controls
takeDamage(dmg: Int)	Decreases player health by the specified damage amount
collectLoot(loot: Loot)	Transfers discovered loot items into the player's inventory
buyItem(item: Item)	Player attempts to buy specified item at the shop
submitAnswer(answer: String)	Provides the player's solution attempt to a mathematical problem
eatFood()	Player eats food to restore their health
repairRaft()	Player uses wood to repair the raft
Relationships	
Has 1 Inventory - Player maintains a personal inventory for storing items and resources	
Initiates 1 or more SeaTravel - Whenever a player enters their ship, they have the ability to travel to other islands if they want to	

Class: Inventory	
Description	Manages storage and organization of all items and resources collected by the player.
Attributes	
Operations	
insert(item: Item, amount: Int)	Inserts items into inventory storage
remove(item: Item, amount: Int)	Removes specified items from inventory
Relationships	
Contains 0 or more Items - Inventory stores various item types acquired during gameplay	

Class: GameEngine	
Description	Central controller managing overall game state, level progression, and system coordination.
Attributes	

Name	Description
currentIsland: Int	Tracks which island the player is currently on
islandsVisited: Int	Tracks how many islands have been visited
enemiesDefeated: Int	Tracks how many enemies have been defeated
currentDay: Int	Tracks the current day
time: enumeration	Tracks whether it's night or day
unlockedIslands: Map<int, bool>	Tracks which specific islands have been unlocked
Operations	
startGame()	Initializes a new game session with starting conditions
endGame()	Processes game termination and displays final results
saveProgress()	Persists current game state to storage
unlockIsland(island: Island)	Updates which islands have been unlocked
Relationships	
Manages 1 Player - Controls and updates the player entity	
Has 1 Timer - Manages the amount of time spent on a question	

Class: Island	
Description	Represents explorable island locations containing enemies, loot, and challenges.
Attributes	
Name	**Description**
islandID: Int	Unique identifier for each island instance
enemies: List<Enemy>	Collection of hostile entities present on the island
loot: List<Loot>	Available resources and items discoverable on the island
Operations	
islandChallenge(difficulty: Int, choice: String)	Initiates optional mathematical challenges with appropriate difficulty
sharkEncounter()	Triggers a shark encounter when a player fails a question when attempting to unlock an island
generateLoot(difficulty: Int)	Creates loot based on island difficulty and player progression
Relationships	
1 Island has 1 Shop - Main island contains the shop for player to buy items	

Spawns 1 or more Enemies - Islands generate enemy encounters for players
Uses 1 ProblemSystem: Generates problem whenever player engages with zombies

Class: Enemy	
Description	Base class defining common attributes and behaviors for all hostile entities.
Attributes	
Name	**Description**
health: Int	Current health points of the enemy
damage: Int	Amount of damage the enemy inflicts per attack
type: enumeration	Classification of enemy such as zombie or shark
lootTable: Loot	Defines rewards dropped when enemy is defeated
Operations	
attack(damage: int)	Executes an attack against the player or raft
takeDamage(dmg: Int)	Reduces enemy health by specified damage amount
dropLoot(loot: Loot)	Releases loot rewards upon defeat
Relationships	
Drops 1 Loot - Enemies provide loot rewards when defeated	

Class: Shop	
Description	Facilitates equipment purchases.
Attributes	
Name	**Description**
items: List<Equipment>	Inventory of equipment available for purchase
prices: Map<String, Map<String, Int>>	Pricing structure organized by equipment type and tier
Operations	
buyItem(item: Equipment, amount: Int)	Processes purchase transaction for specified equipment
giveItem(item: Equipment)	Transfers purchased equipment to player inventory
displayShop()	Presents available equipment and pricing information

verifyResources(price: Map<string, int>)	Verifies the player has the resources to buy the item
Relationships	
Sells 0 or more Equipment - Shop maintains equipment inventory for player purchases	

Class: ProblemSystem	
Description	Generates and evaluates mathematical problems during gameplay challenges and combat.
Attributes	
Name	**Description**
question: String	Text content of the mathematical problem
answer: Double	Correct solution to the problem
difficulty: Int	Complexity level of the problem
reward: Loot	Reward provided for correct solutions
timeLimit: Int	Maximum seconds allowed for problem completion
Operations	
generateQuestion()	Creates new mathematical problem based on difficulty settings
displayQuestion()	Presents problem to player through interface
checkAnswer(response: String)	Validates player's submitted solution
Relationships	
Rewards 1 Loot - Provides loot rewards for correct problem solutions	

Class: SeaTravel	
Description	Manages raft navigation and random encounter generation during ocean travel.
Attributes	
Operations	
travel(island: Island)	Teleports player to selected island
triggerIslandChallenge()	Triggers the island challenge
Relationships	
Has 1 Island destination - References target island for navigation	
Encounters 0 or more Enemies - May generate shark encounters during travel	
Uses 1 ProblemSystem: Generates problem whenever player engages with sharks	

Class: Loot	
Description	Represents collectible items obtained from enemies or island exploration.
Attributes	
Name	**Description**
items: List<Item>	Collection of items contained in loot
Operations	
drop()	Makes loot available for player collection
Relationships	
References 1 or more Items - Loot packages contain one or more items	

Class: Item <Interface>	
Description	Abstract base class defining common properties for all collectible items.
Attributes	
Name	**Description**
name: String	Item's name
itemType: enumeration	Classification of item category
Operations	
Relationships	

Class: Resource	
Description	Consumable materials used for crafting, repairs, and purchases.
Extends	Item
Attributes: Inherits from Item	
Operations: Inherits from Item	
Relationships: Inherits from Item	

Class: Equipment	
Description	Usable gear that improves player capabilities.
Extends	Item
Attributes: Inherits from Item	
Operations: Inherits from Item	
useEquipment()	Activates or equips the equipment item

Relationships: Inherits from Item

Class: Weapon	
Description	Equipment type that increases damage output against enemies.
Extends	Equipment
Attributes: Inherits from Equipment	
damage: Int	Additional damage provided by the weapon
Operations: Inherits from Equipment	
Relationships: Inherits from Equipment	

Class: Armor	
Description	Equipment type that reduces incoming damage to the player.
Extends	Equipment
Attributes: Inherits from Equipment	
protection: Int	Amount of damage reduction provided
Operations: Inherits from Equipment	
Relationships: Inherits from Equipment	

Class: Upgrade	
Description	Permanent enhancements providing lasting benefits like improved visibility
Extends	Equipment
Attributes: Inherits from Equipment	
Operations: Inherits from Equipment	
Relationships: Inherits from Equipment	

4.3 Sequence Diagrams

The following sequence diagrams illustrate two critical processes within SurviveX: completing island challenges and purchasing equipment. These diagrams show the flow of interactions between objects during specific gameplay scenarios.

4.3.1 Island Challenge Sequence

Figure 3 depicts the process when a player travels to a new island and encounters an island challenge to unlock the island. The interaction begins when the player discovers a new island. The Island then triggers the island challenge to the ProblemSystem, which creates an appropriate mathematical question. The Timer begins tracking response time as the problem displays to the player. Upon answer submission, the system evaluates correctness while checking elapsed time. Success within the time limit unlocks the island for travel. Not solving the problem triggers a shark encounter.

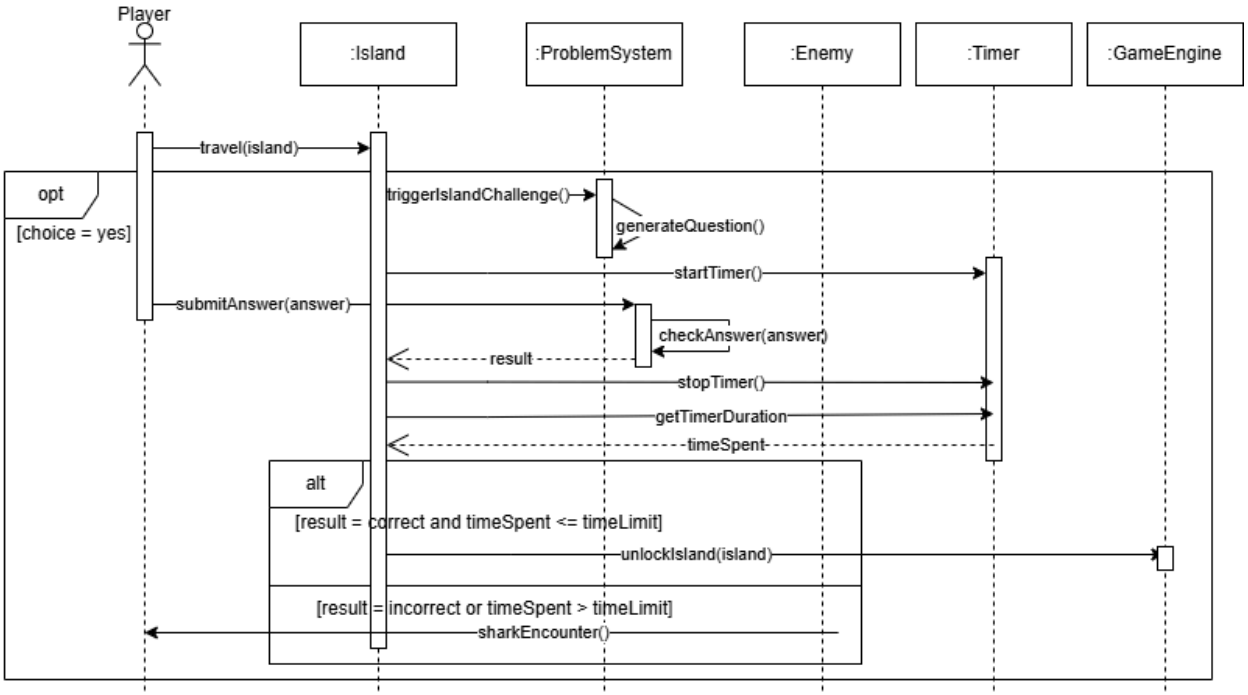

Figure 3: Island Challenge Sequence Diagram

4.3.2 Shop Transaction Sequence

Figure 4 illustrates the purchase process at the shop when a player attempts to buy an item from the shop. The player starts by walking near the shop to interact with it, prompting the GameEngine to display available inventory. The player may choose multiple items in an iterative loop, selecting items to view the information of. When the player decides to purchase an item, the Shop verifies the player has enough resources available. If the player has sufficient resources, the Shop deducts the purchase cost from the Inventory and transfers the equipment to the player. Insufficient resources result in purchase cancellation with appropriate notification to the player.

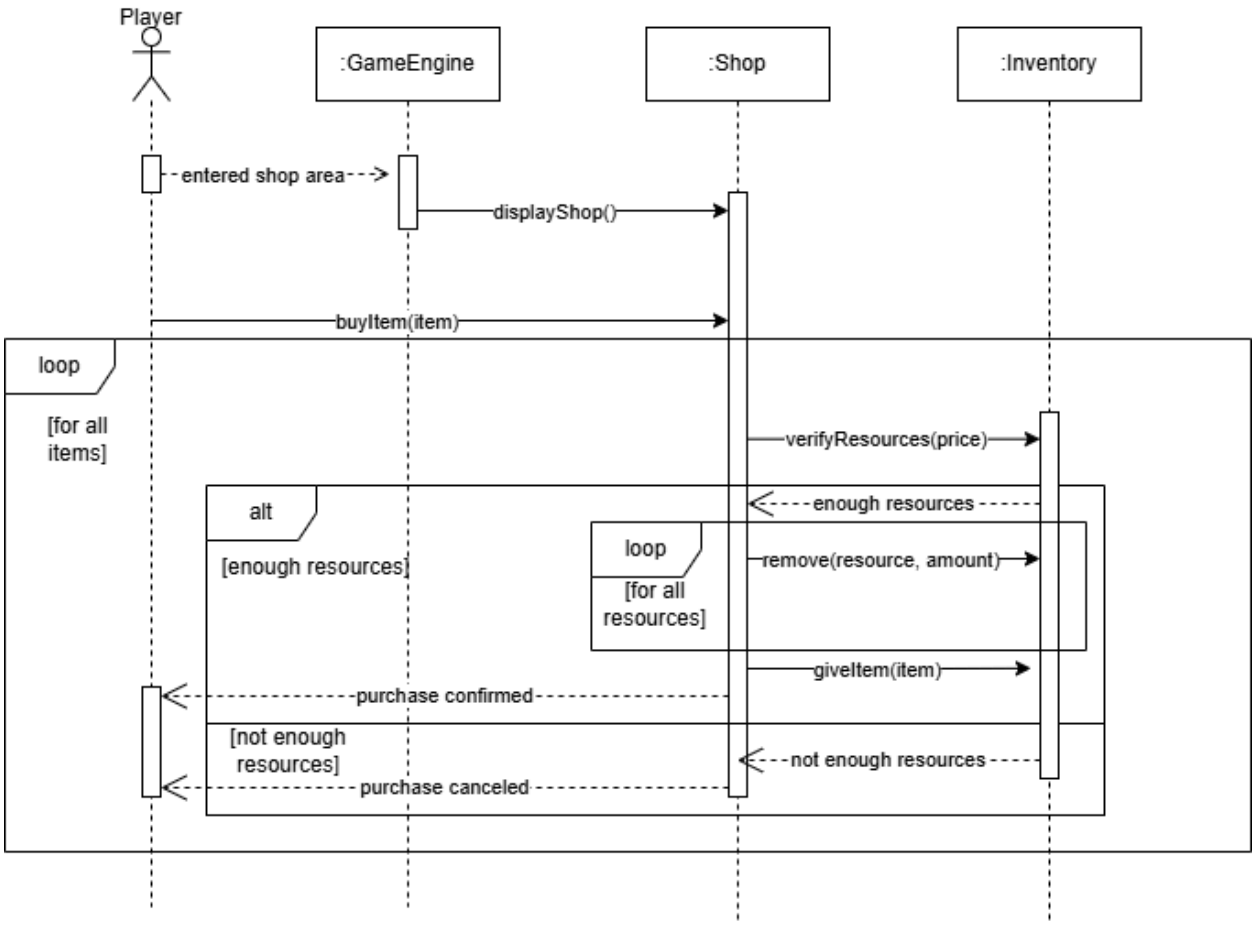

Figure 4: Buy Item Sequence Diagram

4.4 State Diagram

Figure 5 represents the complete flow of game states that players experience during a SurviveX session. The diagram uses standard state diagram notation where filled circles indicate start points, double-ringed circles represent end states, rounded rectangles denote stable states, diamonds indicate decision points, and arrows show transitions between states with their triggering conditions labeled.

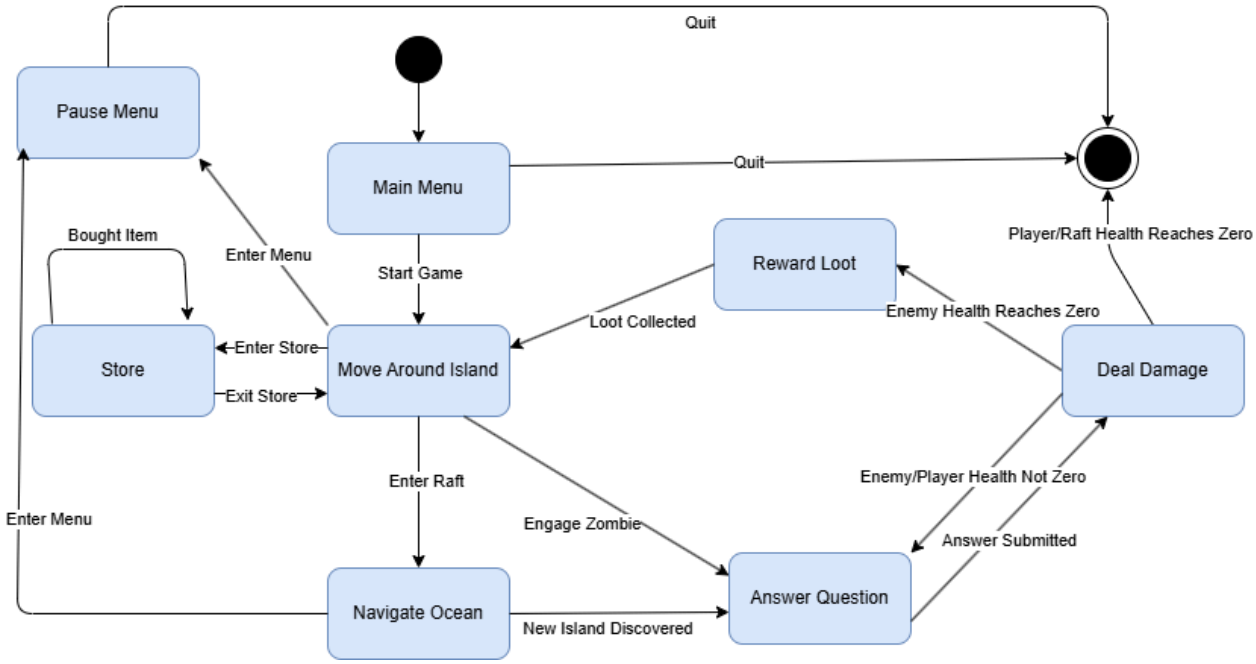

Figure 5: State Diagram

State Flow Overview

Gameplay initiates at the start node and immediately transitions to the Main Menu state, which serves as the entry point for all game sessions. From the Main Menu, players can either begin playing by transitioning to the Move Around Island state or terminate the application by selecting quit, which leads to the end state.

The Move Around Island state functions as the primary gameplay hub where players have the most options available. This state represents the player exploring an island. From this central state, players can engage with several different gameplay elements: they can enter the Store to purchase equipment, participate in zombie combat on islands, navigate the ocean between locations, or answer challenge questions when prompted on islands.

When players choose to engage with challenges or enemies, they transition to the Answer Question state. This state represents the core educational component where mathematical problems are presented. The outcome of problem-solving determines the next state: correct answers deal damage to the opponent, and incorrect answers deal damage to the player. Once the enemy's health reaches zero, the state goes to the Reward Loot state where players receive resources and items, while incorrect answers or timeout situations cause damage, transitioning to the Deal Damage state. If the player or raft's health reaches zero, the game is over.

The Navigate Ocean state handles transitions between islands and manages the day/night cycle. Players can discover new islands, which triggers an island challenge and they proceed to problem-solving.

Combat situations, whether against sharks or zombies, follow similar patterns through the Answer Question state. Success in combat, answering correctly, leads to rewards and continued gameplay, while failure, answering incorrectly, inflicts damage. The Deal Damage state evaluates whether player or raft health has reached zero. If it has, the game transitions to the terminal end state. If health remains above zero, players return to the Move Around Island state to continue their session.

The Store state allows equipment browsing and purchasing. After completing transactions, players return to the Move Around Island state with their newly acquired items. The Pause Menu state is accessible from the Main Menu and provides players with options to quit the game, leading to the end state.

5 Prototype

The prototype of SurviveX will include a main menu, a day/night cycle, and four islands for the player to move around and explore in. Upon starting the game, the player will spawn on the first island which contains a shop where you can buy equipment with resources obtained from island challenges, chest, and defeating zombies. To fight a zombie, the player will have to run into it, and then answer math questions to deal damage to it. Incorrectly answering will cause the player to take damage. The player can restore health by eating food.

After defeating every zombie on an island, a chest will spawn (from island 2 onwards) and the player will unlock the next island. To travel to other islands, the player can go to the dock. At the dock, the player can choose which unlocked island they want to go to. If the island the player is going to is unexplored by the player, then the player will first have to answer a slope question before travelling to the island. If the players gets the slope question wrong, then they will have to fight a shark. The player has to answer math questions to deal damage to the shark and their raft will take damage if the answer is incorrect. Rafts can be repaired using wood. Upon reaching a new island, the player will have the option of taking on an island challenge.

The game will be successfully completed when the player defeats every zombie on every island. However, if player or raft health reaches 0, then it's game over.

5.1 How to Run Prototype

The SurviveX prototype runs entirely in a web browser and requires no downloads or installations. To play the game, follow these steps:

To play, simply go to the SurviveX website (link here: SurviveX), click on the *Play Now* button (colored green). From there you should see a **Prototype** section with a link indicating to play the game prototype within the browser. Once you click that link, wait a few seconds for the game to load in a separate web page. When the main menu appears, click Start and the game will begin. A 30-second timer will start automatically, and enemies will appear on the map. When you encounter an enemy, a math question will pop up. Answer correctly to defeat the enemy, or the game will end if the answer is wrong or time runs out.

5.2 Sample Scenarios

When you open the SurviveX website, the game loads right in your browser.

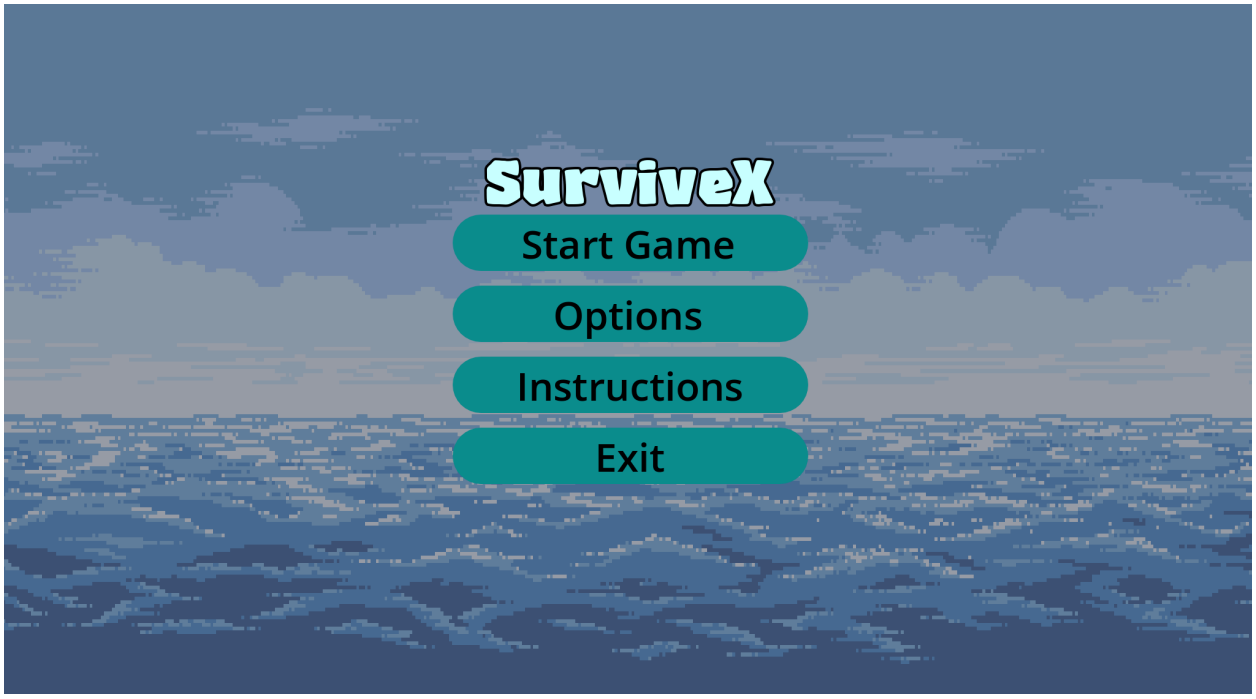

Figure 6: Main Menu Screen of SurviveX which is shown when you open the game. The player can start the game, view options, view instructions, or exit the game.

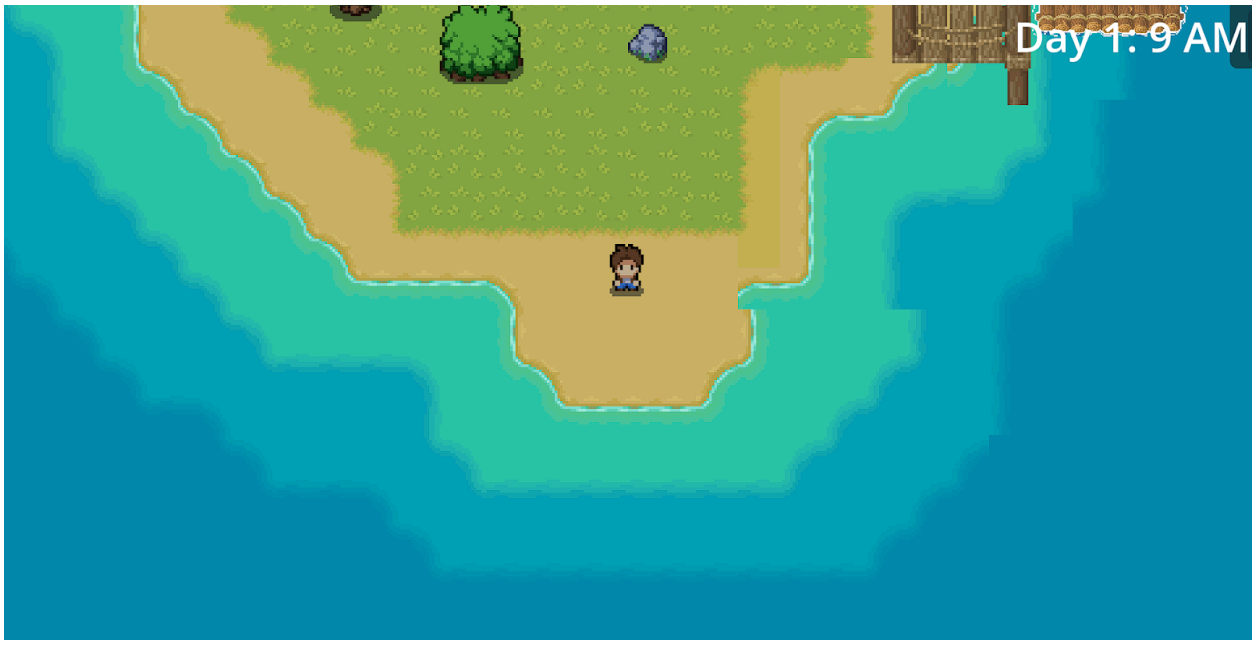

Figure 7: The player spawns on the first island. The date and time can be viewed in the top right corner.

Figure 8: The player fights a zombie by solving math questions.

Figure 9: The player answers incorrectly and takes some damage.

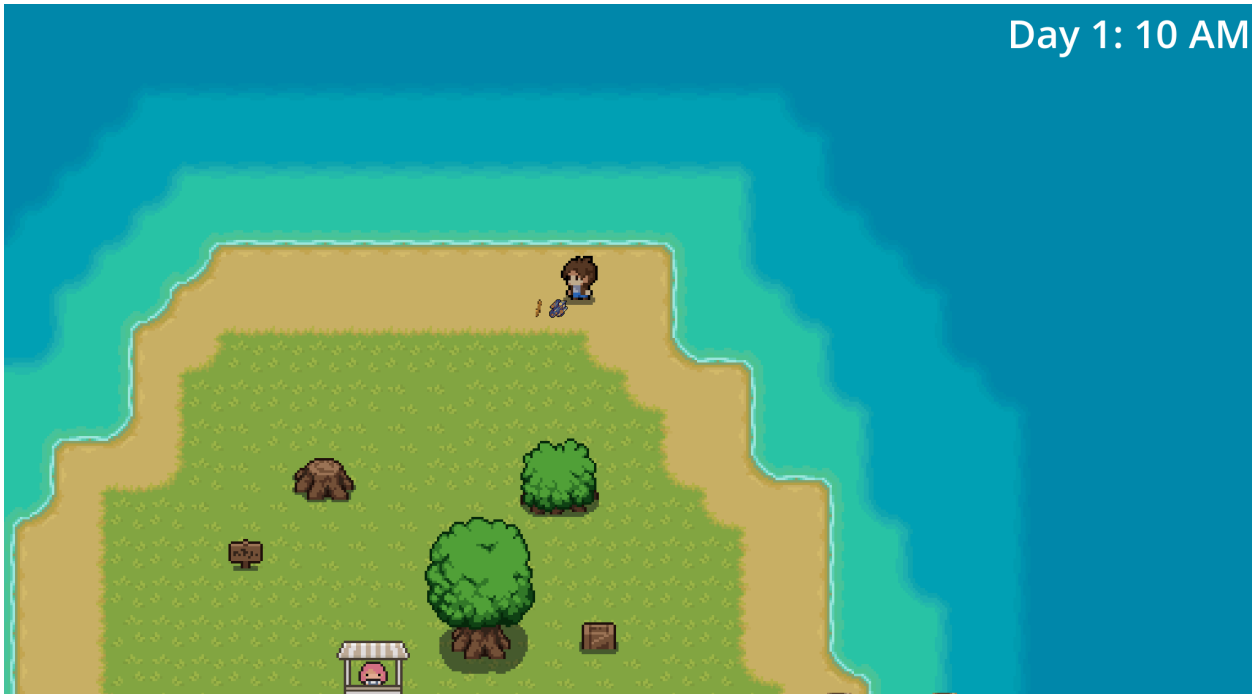

Figure 10: The player successfully defeats the zombie which dropped some resources.

Figure 11: The player accessing their inventory

Figure 12: The player looking through the shop to see what they can buy

Figure 13: The player selecting which island to travel to

Figure 14: The player solving a solve problem to unlock the next island

6 References

[1] *Massachusetts Department of Elementary and Secondary Education Current Framework,* Massachusetts Department of Education, 2025, https://www.doe.mass.edu/frameworks/current.html

[2] *draw.io Documentation*, draw.io., 2025, https://www.drawio.com/doc

[3] *React Documentation*, React, 2025, https://react.dev/learn

[4] *Next.js Docs*, Next.js, 2025, from https://nextjs.org/docs

[5] *How to Create an RPG in Godot 4 (step by step)*, April 3rd, YouTube, 2025, https://www.youtube.com/watch?v=pBoXqW4RykE&list=PL3cGrGHvkwn0zoGLoGorwvGj6dHCjLaGd&index=1

[6] *Game Water tile set by SciGho*, itch.io., 2025*: https://ninjikin.itch.io/water*

[7] *Game pixel art tile set by Game Endeavor* , itch.io., 2025*: https://game-endeavor.itch.io/mystic-woods*

[8] *Game zombie asset pack by TheLazyStone,* , itch.io., 2025*: https://thelazystone.itch.io/post-apocalypse-pixel-art-asset-pack*

[9] *Game Shark asset pack by Sevarihk, OpenGameArt.org*., 2025: *.\https://opengameart.org/content/shark-sprites-animated-4-directional*

[10] *Game Night Vision item asset pack by Andrey Kalyuzhnyy,* itch.io, 2025: https://antahonist.itch.io/pixel-spell-icons-pack-32x32

[11] *Game Armor asset pack by Ponk,* itch.io, 2025: https://jeevo.itch.io/anvil-icons

[12] *Game Weapon asset pack by Luca Pixel Graphics: https://www.patreon.com/LucaPixel,* itch.io, 2025: https://lucapixel.itch.io/weapons-kit-pixel-art

7 Point of Contact

For further information regarding this document and project, please contact **Prof. Daly** at University of Massachusetts Lowell (james_daly at uml.edu). All materials in this document have been sanitized for proprietary data. The students and the instructor gratefully acknowledge the participation of our industrial collaborators.